COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***SQE Lab***
Instructor	***Mam Munazza***
Assignment No.	***04***

Registration No.	***Name***
FA24-BSE-043	***Sohaib Wali***
FA24-BSE-044	***Syed Ahmed Hassan Bukhari***
FA24-BSE-063	***Muhammad Fahad***

1. Introduction

System Design

System Overview

The Student Management System is a simple Python-based project developed to manage student records efficiently. The system allows users to add, search, update, view, and delete student information. It also generates reports based on student marks and grades. The project was developed using basic programming concepts such as functions, loops, and conditional statements.

3.2 Core Functionalities

The main functionalities of the system include:

1. Add Student Registers new students with roll number, name, and marks in three subjects (Math, English, Science)

2. View Students Displays all student records with their marks and calculated averages

3. Search Student Finds specific student by roll number and shows their complete details

4. Update Marks Allows modification of existing student marks

5. Delete Student Removes student records from the system

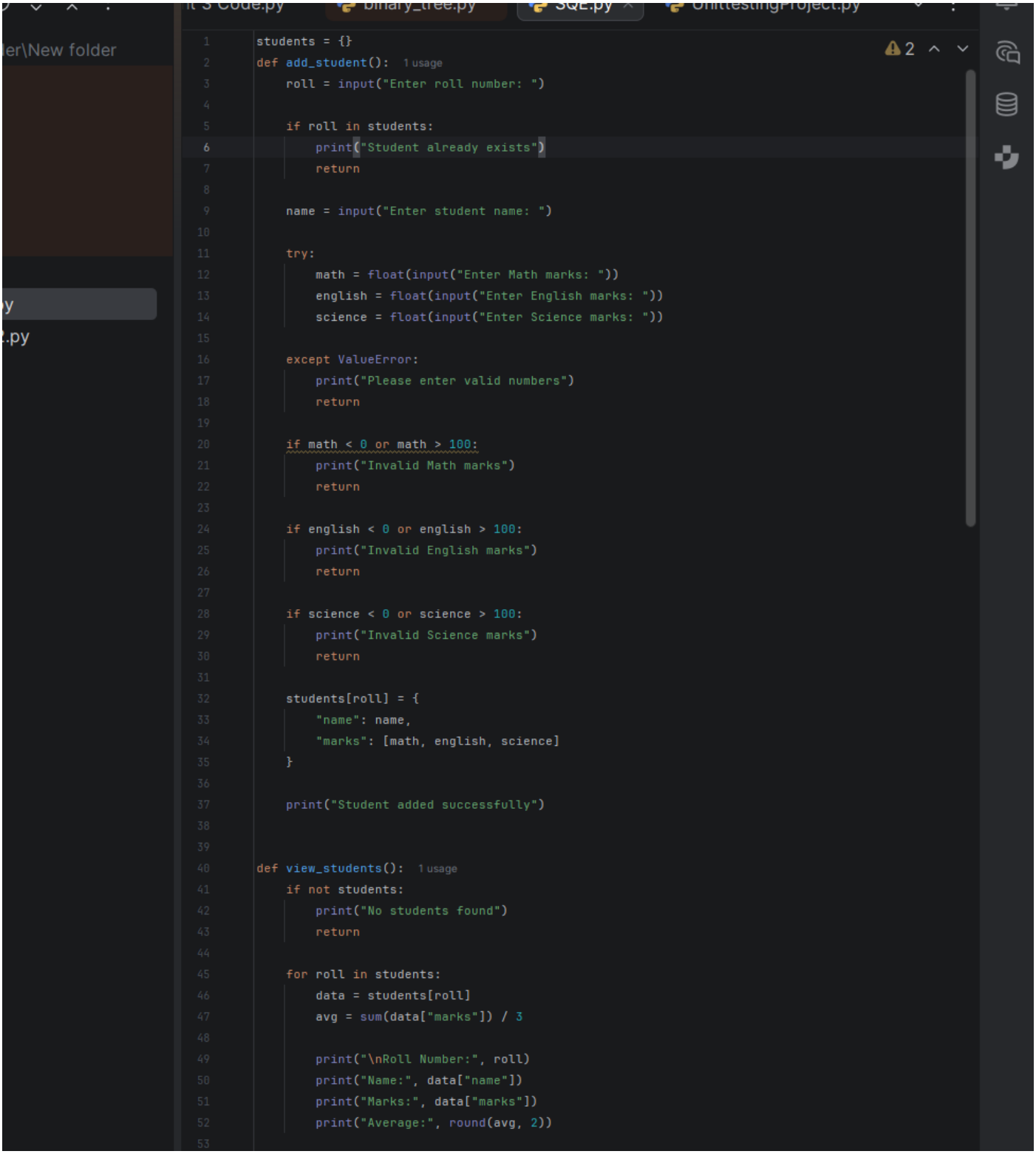
6. Generate Report Creates grade-based reports (A, B, C, F) for all students

The system validates user input at every step, including:

- Checking for duplicate roll numbers
- Ensuring marks are numeric values
- Verifying marks are within valid range (0-100)
- Handling empty inputs and invalid choices

4. Implementation

Source Code



```python
students = {}
def add_student():  1 usage
    roll = input("Enter roll number: ")

    if roll in students:
        print("Student already exists")
        return

    name = input("Enter student name: ")

    try:
        math = float(input("Enter Math marks: "))
        english = float(input("Enter English marks: "))
        science = float(input("Enter Science marks: "))

    except ValueError:
        print("Please enter valid numbers")
        return

    if math < 0 or math > 100:
        print("Invalid Math marks")
        return

    if english < 0 or english > 100:
        print("Invalid English marks")
        return

    if science < 0 or science > 100:
        print("Invalid Science marks")
        return

    students[roll] = {
        "name": name,
        "marks": [math, english, science]
    }

    print("Student added successfully")


def view_students():  1 usage
    if not students:
        print("No students found")
        return

    for roll in students:
        data = students[roll]
        avg = sum(data["marks"]) / 3

        print("\nRoll Number:", roll)
        print("Name:", data["name"])
        print("Marks:", data["marks"])
        print("Average:", round(avg, 2))
```

```python
def search_student():  1 usage
    roll = input("Enter roll number: ")

    if roll in students:
        data = students[roll]
        avg = sum(data["marks"]) / 3

        print("\nStudent Found")
        print("Name:", data["name"])
        print("Marks:", data["marks"])
        print("Average:", round(avg, 2))

    else:
        print("Student not found")


def update_marks():  1 usage
    roll = input("Enter roll number: ")

    if roll not in students:
        print("Student not found")
        return

    try:
        math = float(input("Enter new Math marks: "))
        english = float(input("Enter new English marks: "))
        science = float(input("Enter new Science marks: "))

    except ValueError:
        print("Please enter valid numbers")
        return

    if math < 0 or math > 100:
        print("Invalid Math marks")
        return

    if english < 0 or english > 100:
        print("Invalid English marks")
        return

    if science < 0 or science > 100:
        print("Invalid Science marks")
        return

    students[roll]["marks"] = [math, english, science]
    print("Marks updated successfully")


def delete_student():  1 usage
    roll = input("Enter roll number: ")

    if roll in students:
        del students[roll]
        print("Student deleted successfully")
    else:
        print("Student not found")


def generate_report():  1 usage
    if not students:
        print("No students found")
```

Function Descriptions

add_student()

Adds a new student record with roll number, name, and marks.

view_students()

Displays all student records with their averages.

search_student()

Searches and displays a student using roll number.

update_marks()

Updates marks of an existing student.

delete_student()

Deletes a student record from the system.

generate_report()

Generates a report showing student averages and grades.

Test Cases

Test Case 1

Test ID: TC001

Description: Add student with valid data

Steps:

1. Select option 1
2. Enter roll number
3. Enter student name
4. Enter valid marks

Expected Result: Student added successfully

Actual Result: Student added successfully

Status: PASS

Test Case 2

Test ID: TC002

Description: Add student with duplicate roll number

Steps:

1. Add a student
2. Select option 1 again
3. Enter same roll number

Expected Result: Student already exists

Actual Result: Student already exists

Status: PASS

Test Case 3

Test ID: TC003

Description: Enter invalid marks above 100

Steps:

1. Select option 1
2. Enter marks greater than 100

Expected Result: Invalid marks message displayed

Actual Result: Invalid marks message displayed

Status: PASS

Test Case 4

- **Test ID: TC004**
- **Description: Enter text instead of marks**
- **Steps:**
    1. Select option 1
    2. Enter text like "ABC" in marks field

Expected Result: Please enter valid numbers

Actual Result: Please enter valid numbers

Status: PASS

Test Case 5

Test ID: TC005

Description: Search existing student

Steps:

1. **Add a student**
2. **Select option 3**
3. **Enter existing roll number**

Expected Result: Student details displayed

Actual Result: Student details displayed

Status: PASS

Test Case 6

Test ID: TC006

Description: Update student marks

Steps:

1. **Add a student**
2. **Select option 4**
3. **Enter new marks**

Expected Result: Marks updated successfully

Actual Result: Marks updated successfully

Status: PASS

Test Case 7

Test ID: TC007

Description: Delete existing student

Steps:

1. Add a student
2. Select option 5
3. Enter roll number

Expected Result: Student deleted successfully

Actual Result: Student deleted successfully

Status: PASS

Test Case 8

Test ID: TC008

Description: Generate student report

Steps:

1. **Add students**
2. **Select option 6**

Expected Result: Student report with grades displayed

Actual Result: Student report displayed correctly

Status: PASS

Test Case 9

Test ID: TC009

Description: Search non-existing student

Steps:

1. Select option 3
2. Enter roll number that does not exist

Expected Result: Student not found

Actual Result: Student not found

Status: PASS

Test Case 10

Test ID: TC010

Description: Invalid menu option

Steps:

1. Run the program
2. Enter invalid menu choice like 9

Expected Result: Invalid choice message displayed

Actual Result: Invalid choice message displayed

Status: PASS

5.2 Unit Tests:

```python
import unittest
import SQE as student_system
class TestStudentSystem(unittest.TestCase):

    def setUp(self):
        student_system.students.clear()

    def test_add_student(self):
        student_system.students['101'] = {"name": "Ali", "marks": [85, 90, 88]}
        self.assertEqual(student_system.students['101']['name'], second: "Ali")
        self.assertEqual(len(student_system.students), second: 1)

    def test_duplicate_student(self):
        student_system.students['101'] = {"name": "Ali", "marks": [85, 90, 88]}
        self.assertIn( member: '101', student_system.students)
        self.assertEqual(len(student_system.students), second: 1)
    def test_average_calculation(self):
        marks = [60, 70, 80]
        avg = sum(marks) / 3
        self.assertEqual(avg, second: 70.0)
    def test_grade_A(self):
        avg = 85
        if avg >= 80:
            grade = "A"
        self.assertEqual(grade, second: "A")

    def test_grade_B(self):
        avg = 70
        if avg >= 65:
            grade = "B"
        else:
            grade = "C"
        self.assertEqual(grade, second: "B")
    def test_grade_C(self):
        avg = 55
        if avg >= 50:
            grade = "C"
        else:
            grade = "F"
        self.assertEqual(grade, second: "C")

    def test_grade_F(self):
        avg = 40
        if avg >= 50:
            grade = "C"
        else:
            grade = "F"
        self.assertEqual(grade, second: "F")

    def test_delete_student(self):
        student_system.students['101'] = {"name": "Ali", "marks": [85, 90, 88]}
        del student_system.students['101']
        self.assertNotIn( member: '101', student_system.students)

    def test_update_marks(self):
        student_system.students['101'] = {"name": "Ali", "marks": [85, 90, 88]}
        student_system.students['101']['marks'] = [75, 80, 85]
        self.assertEqual(student_system.students['101']['marks'], second: [75, 80, 85])

if __name__ == '__main__':
    unittest.main()
```

5.3 Unit Test Results:

```
(.venv) PS D:\New folder\New folder> python -m unittest UnittestingProject.py
.........
----------------------------------------------------------------------
Ran 9 tests in 0.001s

OK
(.venv) PS D:\New folder\New folder>
```

6. Bug Reporting

Bugs Found

Bug 1

Bug ID: BUG001 **Bug Name:** Program crashes on non-numeric input for marks **Description:** When user enters text instead of numbers for marks, program crashes **Steps to Reproduce:**

1. Select "Add Student"
2. Enter roll number
3. Enter name
4. Enter "abc" for Math marks

Expected Result: Show error message

Actual Result: Program crashed with Value Error

Severity: High

Status (Pass/Fail): Pass

Bug 2

Bug ID: BUG002 **Bug Name:** Accepts marks greater than 100 **Description:** System accepts marks like 150 which is invalid **Steps to Reproduce:**

1. Add student
2. Enter 150 for any subject

Expected Result: Reject with error message

Actual Result: Accepted the marks

Severity: Medium

Status (Pass/Fail): Pass

Bug 3

Bug ID: BUG003 **Bug Name:** Accepts negative marks **Description:** System accepts negative marks like -20 **Steps to Reproduce:**

1. Add student
2. Enter -20 for marks

Expected Result: Reject with error

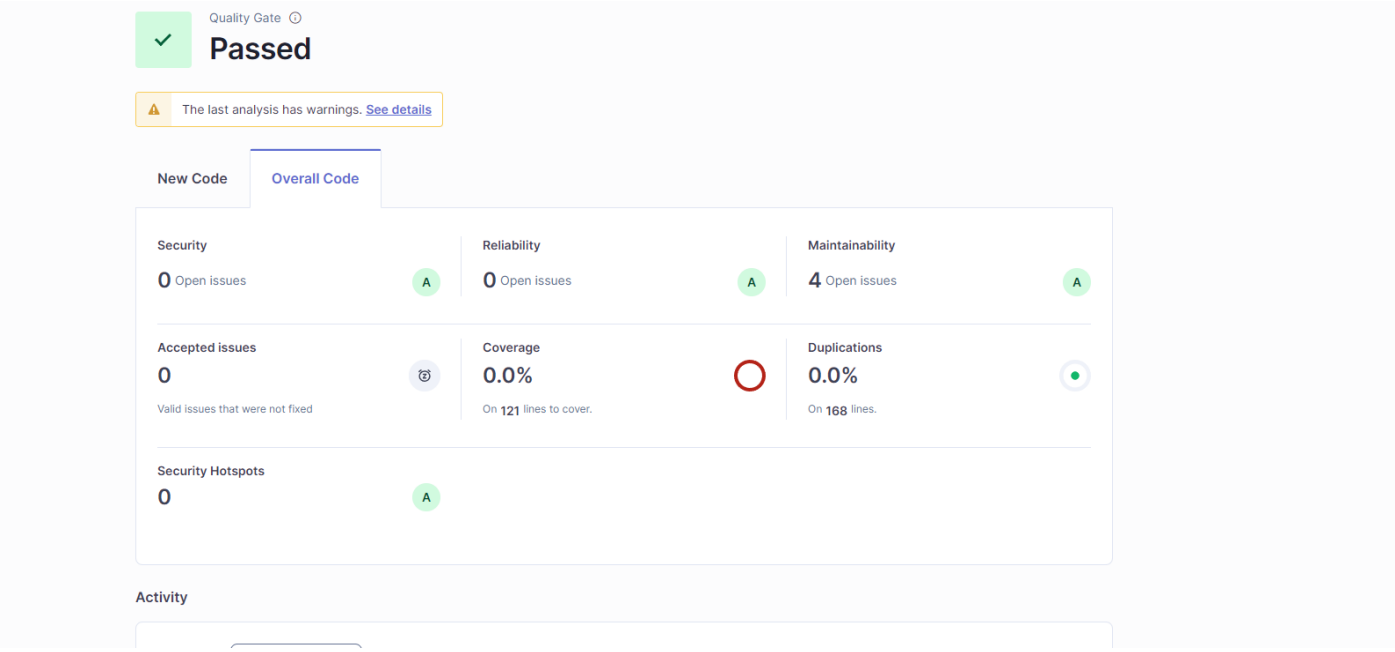
Actual Result: Accepted negative value

Severity: Medium

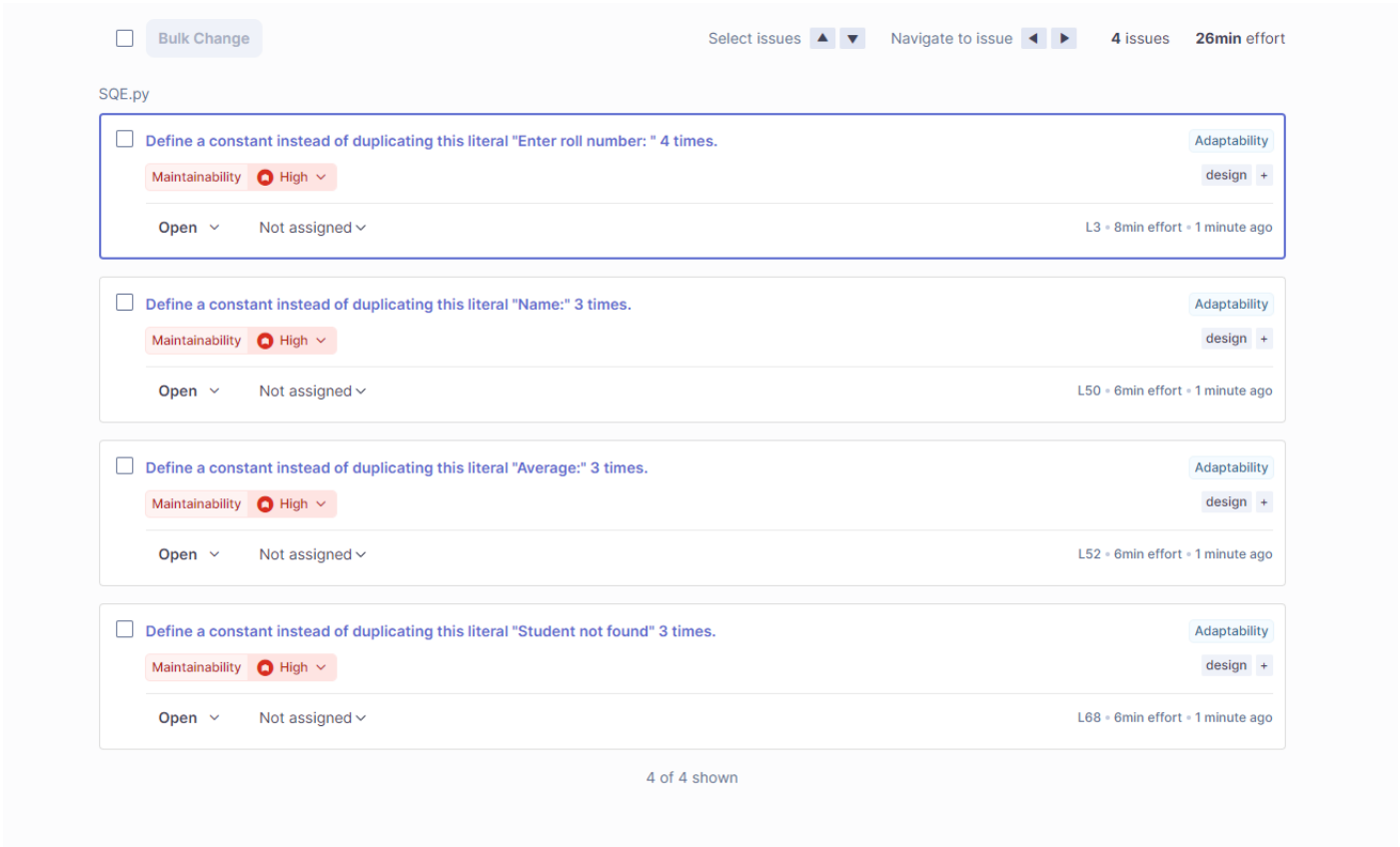
Status (Pass/Fail): Pass (FIXED - validation added)

Code Quality Analysis SonarQube

Results:

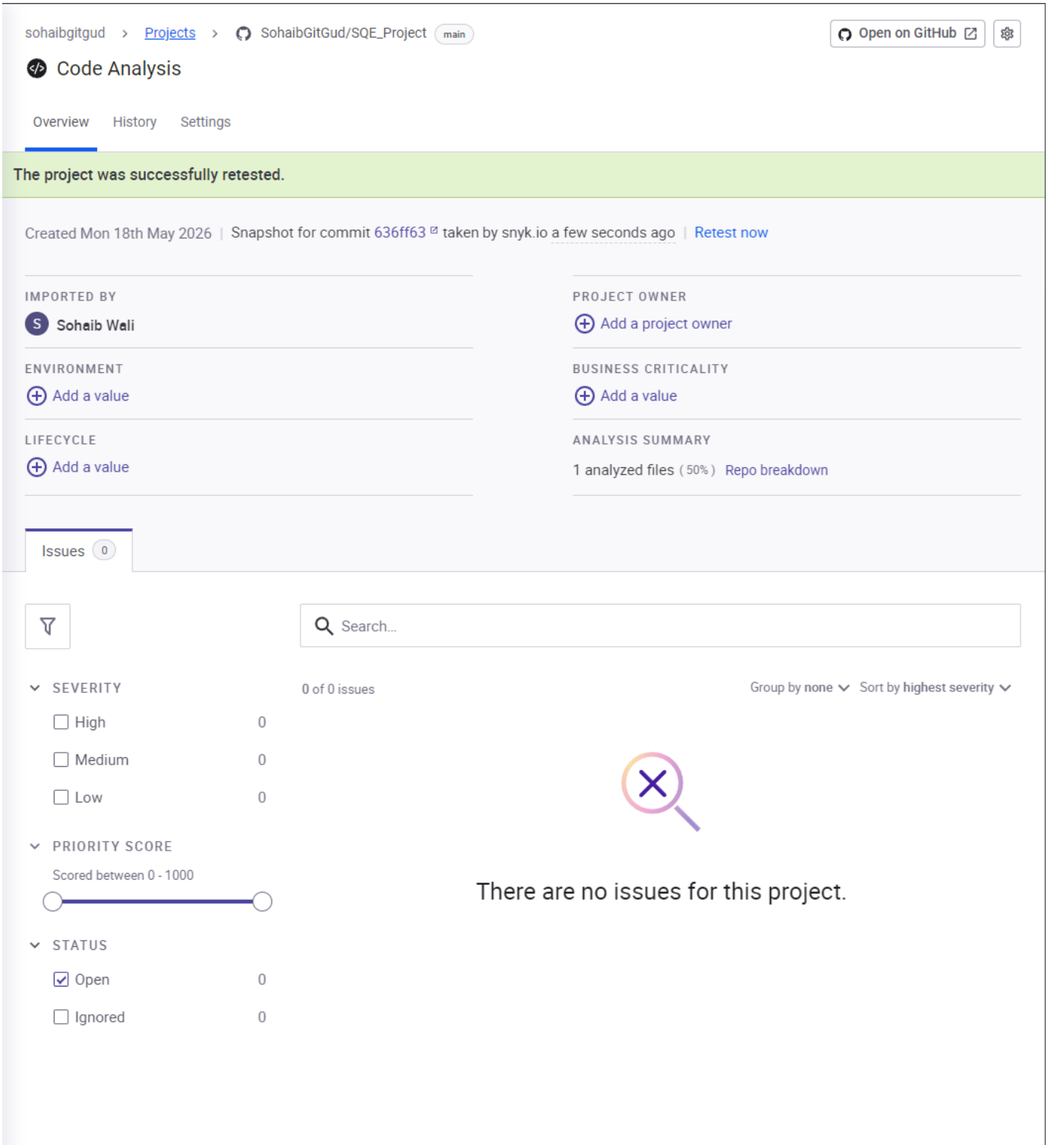


Issues Identified:



8. Security Analysis – Snyk (Phase 5)

8.2 Scan Results



Manual Security Analysis:

1. No Input Validation for Roll Numbers

- System accepts any text as roll number
- Risk: Possible harmful attacks if database added later
- Fix: Use regex to allow only letters and numbers

2. No Authentication System

- Anyone can access and change student data without login
- Risk: Unauthorized access and data modification
- Fix: Add login system with user roles

3. Plain Text Data Storage

- Student data stored in memory without encryption
- Risk: Sensitive information could be exposed if hacked
- Fix: Use encryption or secure database

a

GitHub Repository Link:

https://github.com/SohaibGitGud/SQE_Project1